# Übungsblatt 9: Multicore & Finale (Klausurvorbereitung)

**Mikrocomputertechnik 3 (MCT3)**

**Musterlösung**

## Zielsetzung

Sie vertiefen Vorlesung 9: Cache-Kohärenz / MESI, Race Conditions und RISC-V-Atomics (LR/SC) sowie eine kurze Klausur-Selbstkontrolle über Speicher, Interrupts, PWM und DMA.

**Bearbeitungszeit:** ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 8 P, Aufgabe 3: 8 P (Gesamt: 24 P)

## Teil 1: Theorie — Cache-Kohärenz und MESI

### Aufgabe 1.1: Veraltete Daten im SMP

In einem SMP-SoC hat jeder Kern einen privaten L1-Cache mit *Write-Back*.

1. Beschreiben Sie das Cache-Kohärenz-Problem, wenn Core 0 eine Variable ändert und Core 1 sie kurz darauf lesen will. Warum darf Core 1 nicht einfach den gemeinsamen RAM lesen?
2. Warum löst man das Problem per Hardware (Bus Snooping) statt durch manuelles Software-Flushing vor jedem Lesevorgang?

**Musterlösung Aufgabe 1.1**

1. Write-Back hält die neue Kopie zunächst im privaten L1. Core 1 darf den RAM nicht als einzige Wahrheit lesen. Write-Through ohne Invalidation würde den RAM aktualisieren und die fremde Cache-Kopie trotzdem stehen lassen.
2. Für häufiges cachebares Sharing ist manuelles Spülen unpraktikabel. Non-cacheable Speicher bleibt eine AMP-Möglichkeit. Hardware-Kohärenz kostet Traffic, auch wenn die Anwendung sie nicht aufruft.

### Aufgabe 1.2: MESI-Protokoll

1. Nennen Sie die vier MESI-Zustände und ihre Bedeutung.
2. Core 0 und Core 1 haben dieselbe Line beide in **S (Shared)**. Was passiert auf dem Bus, wenn Core 0 schreiben will? Welche Zustände haben die Lines danach in Core 0 und Core 1?

### Musterlösung Aufgabe 1.2

1. **M:** geändert und exklusiv, unterer Stand kann alt sein. **E:** sauber und exklusiv. **S:** sauber und teilbar, nicht zwingend mehrere nachweisbare Kopien. **I:** lokal ungültig.
2. Core 0 erwirbt Ownership und schließt die Invalidates ab. Core 1 geht nach **I**, Core 0 nach **M**. Der Broadcast allein ist noch nicht der Write.

## Teil 2: Theorie — Race Conditions und Hardware-Atomics

### Aufgabe 2.1: Anatomie eines Software-Fehlers

Betrachten Sie `zaehler++;` auf einer globalen Variable.

1. Übersetzen Sie den Befehl logisch in die drei RMW-Schritte (Assembler-Ebene).
2. Erklären Sie chronologisch eine Race Condition: beide Kerne inkrementieren „gleichzeitig“, das Ergebnis steigt aber nur um 1 statt um 2 (Startwert 0).

**Musterlösung Aufgabe 2.1**

1. `lw` (lesen) → `addi` (+1) → `sw` (zurückschreiben).
2. Im Hardwarebild lesen beide 0 und schreiben beide 1. Ein Update geht verloren. Ein normales konkurrierendes C-`int` ist außerdem ein Data Race und in C11 undefiniert. `volatile` macht daraus keine Synchronisation.

### Aufgabe 2.2: RISC-V Atomics (LR/SC)

Software-Locks wie `while(lock==1); lock=1;` scheitern, weil Prüfen und Setzen selbst racy sind. RISC-V nutzt `lr.w` / `sc.w`.

1. Erklären Sie `lr.w` und `sc.w`. Welche Rolle hat der Hardware-Monitor?
2. Was passiert, wenn Core 1 `sc.w` ausführt, nachdem Core 0 die Adresse bereits verändert hat?

**Musterlösung Aufgabe 2.2**

1. `lr.w` liest und setzt eine Reservation auf eine implementierungsabhängige Granule, nicht auf ein gehaltenes Mutex. `sc.w` schreibt nur bei gültiger Reservation.
2. Der Store unterbleibt. Der Rückgabewert ist ungleich 0; Erfolg wäre 0. Die Software wiederholt ab `lr.w`. Ein Fehlschlag ist auch ohne Store eines anderen Harts zulässig. DMA bricht die Reservation nur, wenn die Plattform dieselbe Domäne definiert. Für ein Schloss kommen Acquire und eine Release-Fence hinzu.

## Teil 3: Systemverständnis (Klausur-Wrap-Up)

*Kurze Selbstkontrolle quer über das Semester.*

### Aufgabe 3.1: Speicher und Caches

1. Warum führt Column-Major-Zugriff auf ein 2D-Array in C zu starkem Thrashing, Row-Major aber nicht?
2. Unterschied Direct Mapped vs. 4-Way Set-Associative bei der Platzwahl eines neuen Blocks?

**Musterlösung Aufgabe 3.1**

1. C speichert zeilenweise (Row-Major). Zeilenweise Zugriffe nutzen räumliche Lokalität (Cache-Line). Spaltenweise Sprünge oft über große Abstände → Nachbarn unnütz, häufiger Miss / Thrashing.
2. Direct Mapped: Index → *ein* fester Slot. 4-Way: Index → Set mit 4 Ways zur Wahl → weniger Konflikt-Misses.

### Aufgabe 3.2: Interrupts und Timer

1. Architektonischer Unterschied CLINT vs. PLIC? Je ein typisches Einsatzgebiet.
2. Warum darf eine ISR nicht wie eine normale C-Funktion mit `ret` enden (`__attribute__((interrupt))`, `mret`)?

**Musterlösung Aufgabe 3.2**

1. **CLINT:** kernnah, geringe Latenz — lokale Timer (`mtime`/`mtimecmp`), Software-/IPI-Interrupts. **PLIC:** bündelt/priorisiert externe Peripherie-IRQs (UART, SPI, GPIO, …) und meldet typisch External Interrupt.
2. ISR unterbricht asynchron; normales `ret` restauriert Kontext/`mstatus` nicht korrekt. Das Attribut erzwingt Prolog/Epilog; Abschluss mit `mret`.

### Aufgabe 3.3: DMA und Aktorik

1. Rollen von `ARR` und `CCR` bei PWM?
2. Was ist ein Interrupt Storm beim SPI-Senden großer Datenmengen per IRQ — und warum hilft DMA?

**Musterlösung Aufgabe 3.3**

1. `ARR` begrenzt die Periode/Frequenz; `CCR` ist die Compare-Schwelle (Duty Cycle, typisch CCR/ARR im Vorlesungsmodell).
2. Ein Byte-IRQ bei schnellem SPI kann die angenommene CPU-Kapazität überschreiten. DMA übernimmt die Kopierschleife und hebt das SCK-Limit nicht auf.

### Aufgabe 3.4: AMAT-Wiederholung

Hit Time = 1 Takt, Miss Rate = 5 %, Miss Penalty = 100 Takte.

1. Geben Sie die AMAT-Formel an und berechnen Sie die AMAT.

**Musterlösung Aufgabe 3.4**

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty}) = 1 + (0{,}05 \times 100) = 1 + 5 = 6 \text{ Takte}$$